

Airframe: project summary

Header-only Wi-Fi fault finding for the Airframe hackathon challenge (IT / Network Engineering, Wireless Infrastructure).

1. The challenge

A fleet of passive sensors captures 802.11 traffic continuously, truncated to protocol headers: 802.11 management and control frames and 802.1X/EAP. Payloads and layer-3 traffic are not visible. The task was to ingest a set of related header-only captures and find out what is going wrong on the air:

- Which clients are failing to connect?
- Where is the channel congested?
- Are nearby sensors looking at the same event?

Findings had to be shown in a dashboard and pitched. A design for thousands of sensors and access points was optional.

The rules shaped the approach:

- Analysis is header-only; nothing may be transmitted.
 - MAC addresses, EAP identities and SSIDs must be masked in the dashboard and the pitch.
 - A single deauthentication during a normal roam is not a failure.
 - One client seen by several sensors is one finding, not several.
-

2. The data

The data covers 8 sensors from 14:24 to 14:54 (Paris time) and 1,118,853 frames. The sensors heard 29 access points (53 networks) and 99 clients.

Each sensor listens on its own channel (36, 40, 44, 48, 149, 153, 157, 161), so no frame is heard by two sensors. Sensors are therefore correlated through the same client, not the same frame.

3. What went wrong

1. Critical: 802.1X logins stall at the Identity step, site-wide.

- All 63 clients on the 802.1X network failed, on 26 APs and all 8 channels, and none ever logged in.
- The APs only ever send EAP Identity requests and never start the login method.
- Every client the sensors could hear answered (129 of 129 audible attempts). So the APs are not getting a reply from the authentication server.
- After three tries, 30 s apart, the AP disconnects the client:
 - 502 attempts ended with reason 23 (802.1X authentication failed).

- 139 were cut short by the rejection wave below.
- No EAP-Success frame appears in the capture.
- Owner: identity / RADIUS team. First check: RADIUS logs and controller-to-RADIUS reachability.
- Not proven: which component failed.

2. Serious: rejection wave from 14:30:53.

- The APs tell 18 clients, on 18 APs, that their session is no longer valid (reason 2), every 11–30 s until the capture ends. That is 1,458 reason-2 frames in total.
- Each client is always rejected by the same AP and never tries another.
- All 18 share one vendor prefix (Intel).
- Owner: WLAN controller team. First check: controller events at 14:30:53, and why only one device group.

3. Explained: the probe storm and retry spike at 14:26–14:28 are symptoms, not a radio problem.

- Probe requests from Intel and Apple devices climb to 414 a minute at 14:27. That happens in step with clients hitting their first login timeout (61 of 63 by 14:28).
- The all-frame retry rate doubles to 9.5%. But probe responses are retried about 31% of the time in every minute, so the rise is a change in traffic mix, not worse radio conditions.
- 24 IoT devices rejoining in the same two minutes add a little more.

4. Medium: channel plan. Channels 36 and 40 carry 5 APs each, while most others carry 3. A channel re-plan is a configuration change.

5. Healthy: the PSK / IoT network kept working. All 31 successful joins were Raspberry Pi devices on the PSK network (23 devices). One device stopped after key message 1 at 14:27 and never retried.

6. Sensor coverage. Sensors 01, 05 and 06 almost never hear a client transmit, only the APs. They are candidates for repositioning.

4. Two independent implementations

- **Wireshark-based pipeline** (`pipeline_tshark/`):
 - Extracts header fields with tshark and masks identifiers first.
 - Rebuilds every join attempt per client across sensors, then runs the detectors and groups findings into network-wide incidents.
 - Includes a Streamlit dashboard.
- **Fast pipeline** (`pipeline_fast/`):
 - A from-scratch parser using only the Python standard library. It analyses all 8 captures in about 4.5 s on one core (2.5 s on two), against about 2.5 minutes with tshark.
 - It has the sharper 802.1X diagnosis (the Identity-step loop), operational alerts with detection delay, owner, next checks and "not proven", and a single-file HTML dashboard.
 - It now applies the same judgment rules as the Wireshark-based pipeline (`findings.csv`) and passes the same 16 scenarios and 16 public captures.

They agree on every headline number:

Headline	Fast pipeline	Wireshark-based
802.1X clients failing, APs affected	63, 26	63, 26
Successful joins	5 confirmed + 26 probable	31
Still in progress at capture end	13	13
Reason-2 disconnect frames	1,317 + 141 = 1,458	1,458
Clients in the rejection wave	18	18
IoT handshake stuck at message 1	1	1
Crowded channels (APs vs median 3)	36, 40 (5 each)	36, 40 (5 each)
IoT reconnect burst	24 attempts, 24 clients, 2 min	24 attempts, 24 clients, 2 min

Recommended demo engine: the fast pipeline with the fixes in `pipeline_fast/CHANGES.md`. Use the Wireshark-based pipeline and its test suite as the cross-check and the regression evidence.

5. Review of the fast pipeline

Fixed in this bundle:

- Raw MACs and the SSID appeared in every output. Identifiers are now masked at parse time by default, keeping only the vendor prefix of each MAC; `--no-mask` is for debugging and the report says so.
- A hard-coded "not spoofed" claim was printed without any check. It has been removed and is now listed as "not proven".
- Attempts ended by the rejection wave were counted as 802.1X failures without comment. They are now split out.
- The vendor table was missing Intel, the largest affected group.
- A single-sensor probe storm was labelled "on every channel".

Also fixed since (details in `pipeline_fast/CHANGES.md`):

- Findings for every detector the Wireshark-based pipeline has: EAP-Failure, wrong-passphrase handshakes, full APs, single-client loops and floods, silent APs, sustained congestion and per-client probe storms. Clients with the same problem at the same time roll up into one parent finding.
- A client leaving on its own, "try again later" (status 30) and missed key messages followed by encrypted traffic are no longer failures.
- WPA1 and WEP joins reach "success".
- It reads pcap, pcapng and gzipped captures with radiotap, plain 802.11, Prism, AVS or PPI headers.

Still open: no check that disconnect frames really come from the AP (spoofing), so floods say "not checked". Congestion is judged from retries only, not airtime.

6. Testing

The Wireshark-based pipeline has 56 tests, and all pass (`results/test_results.txt`). The fast pipeline runs through the same scenarios, privacy checks and public captures: 48 checks, all pass (`results/fast_pipeline_tests.txt`).

- **16 public captures** from the Wireshark and aircrack-ng test suites. They cover WEP, WPA2, PMF, WPA3-SAE, OWE, EAP-TLS and Suite-B; radiotap, plain 802.11 and Prism headers; broken timestamps; and a disassociation flood.
- **16 generated multi-sensor scenarios with known answers**, one per detector and judgment rule. They include roam vs loop, the same event on two sensors, wrong passphrase, full AP, silent AP, congestion, a network-wide login failure, a capture ending mid-join, a client leaving on its own, and "try again later".
- **Privacy tests**: no raw MAC address or SSID may appear in any output.
- **Dashboard tests**: every tab renders on real and synthetic outputs.

The public captures exposed eight real bugs, all fixed:

1. Gzipped captures were skipped.
2. It crashed on captures without a radio header.
3. It crashed on invalid timestamps.
4. It crashed on captures with no join attempts.
5. It reported a false handshake failure when the sensor missed M3/M4 but encrypted traffic followed.
6. It reported false congestion on captures without beacons or ACKs, or only seconds long.
7. It counted a capture ending mid-join, a client leaving on its own, and "try again later" as failures.
8. It called a disassociation flood a loop.

Deliberately disabling roam recognition, cross-sensor merging and the encrypted-data rule made the matching tests fail. So the tests do catch regressions.

Cross-check with new sample cases. Eleven further cases that neither test suite had seen were run through both engines: partial credential rejection, a sensor capture gap, clock skew between sensors, a spoofed flood, ping-pong roaming, a hidden network, a randomised MAC, three simultaneous incidents, a handshake timeout after a successful login, a client already connected before the capture, and the probe-response retry trap. The fast pipeline passed 24/24. The Wireshark-based pipeline passed 19/24 and now passes 24/24 after four fixes (AP grouping, clock skew, roam recognition, sensor gaps), with its 56 tests and its results on the challenge data unchanged. Details in `CROSSCHECK_REPORT.md`.

7. Use case and value

Who uses it:

- Wireless operations watches one merged incident board.
- The identity team receives login-path alerts.
- The controller team receives disconnect waves.

- Plant IT sees which device groups are affected and whether production networks are healthy.

The same half hour, with Airframe (times measured from the captures):

- 14:25:28: critical alert to the identity team, 5 s after the first failed login.
- 14:28: the probe storm and retry spike are tagged as symptoms, so there is no reason to page the RF team.
- 14:31:51: serious alert to the controller team, 58 s after the rejection wave began.

Without such a tool, users usually notice first, tickets reach the helpdesk, and the retry spike sends the RF team after a radio problem that does not exist. This "without" timeline is illustrative.

What faster answers are worth. All figures below except the two cited sources are adjustable assumptions in the dashboard.

- **This incident, as captured:** 63 users locked out, about 30 device-hours in 30 minutes. At \$70 per hour that is about \$2,100. Unnoticed for an 8-hour shift, it would be about \$35,000.
- **Per site per year, default settings:** about \$70,000. This assumes 12 incidents a year and 74 minutes saved per incident (detection 30 → 1 min, diagnosis 60 → 15 min), and nearly all of it is user time.
- **Production exposure is the dominant lever.** Siemens puts an idle automotive line at \$2.3 million per hour. Each 1% of incident time that stops a line would add about \$340,000 per site per year. It is off by default because the IoT network kept working in this capture.
- **Chasing the wrong cause.** In a 2016 ZK Research survey, network administrators reported Wi-Fi as the cause 64% of the time, while it was the root cause only 40% of the time. Here the retry spike pointed at the radio; the cause was the login server.
- **Free fixes:** re-plan channels 36 and 40, and reposition sensors 01, 05 and 06.

What the captures cannot prove: business impact. There is no application data, so impact is modelled, not measured. The affected devices are Intel and Apple clients, most likely laptops and phones.

8. At scale

- **Ingestion:** sensors extract headers at the edge and send compact event records tagged with sensor and site, not raw captures.
- **Fan-in:**
 - Each site correlates its own sensors by masked client ID.
 - It keeps per-minute summaries for everything and raw headers only around incidents, in a flight-recorder style.
 - Under load, it drops routine statistics first and never drops authentication failures.
- **Sensor lifecycle:** a registry of each sensor's location, channel and firmware. Heartbeats tell apart a silent sensor, a quiet one (heartbeat but no frames) and a lagging one. Firmware rolls out in stages.
- **Measured throughput:** about 250,000 frames per second on one core with the fast pipeline (measured on a cloud VM). At this site's traffic of about 78 frames per second per sensor, one core keeps up with roughly 3,000 sensors.

9. Privacy

- Both pipelines mask identifiers before analysis by default and never read EAP identities.
- The dashboard and every file in `results/` contain only masked identifiers.
- The salt files are not part of this bundle; keep them private.
- The original uploaded report (`real_report.txt`) contained raw MAC addresses, so it is not included. `results/fast_pipeline_report.txt` is the same report, with every number identical, produced by the patched pipeline with masked identifiers.
- The captures themselves are not included: they may not leave the approved environment.

Sources

- Siemens, *The True Cost of Downtime 2024*: an idle automotive production line costs \$2.3 million per hour.
- ZK Research, survey of 100 network administrators (2016): Wi-Fi reported as the cause 64% of the time, root cause 40%; most Wi-Fi issues take more than 30 minutes to diagnose and resolve.